A certified doubly nonnegative bound for branch and bound on the quadratic assignment problem, and nug30 revisited

Nathan W. Brixius*

October 2026

Abstract

In 2000, the quadratic assignment problem nug30 was solved by a branch-and-bound algorithm based on a convex quadratic programming bound (QPB), running for about a week on a computational grid of hundreds of workstations. We revisit nug30 with the same QPB-based branch and bound, strengthened near the top of the tree by a much stronger, much more expensive bound: the doubly nonnegative (DNN) semidefinite relaxation of Zhao, Karisch, Rendl and Wolkowicz, in the facially reduced form of Oliveira, Wolkowicz and Xu, solved by ADMM. Because a branch-and-bound proof may only use valid bounds, and an ADMM iterate is not one, we compute at every check a *certified* lower bound: the Lagrangian dual value of a multiplier whose projection onto the relaxation's face is negative semidefinite by construction, evaluated by a Gilmore–Lawler-type minimization over permutations, with a rigorous floating-point safety margin (Rump's Cholesky criterion). We describe the bound, its certificate, an $O(m^4)$-per-product implementation of the facially reduced ADMM with a single-precision subspace eigensolver, and its use in branch and bound, where it is applied only to nodes at depths 2–6 that QPB does not fathom and used only for fathoming. With it, nug24 is solved in 299 nodes and 13 CPU minutes, and nug30 in about three hours on four cores.

1 Introduction

The quadratic assignment problem (QAP) in Koopmans–Beckmann form is

$$\min_{X \in \Pi_n} \ \operatorname{tr}(AXBX^T) + \operatorname{tr}(CX^T), \tag{1}$$

where Π_n is the set of $n \times n$ permutation matrices, A (flows) and B (distances) are $n \times n$, and C is a linear cost. QAPLIB [3] has long served as its benchmark library, and the Nugent instances [7] nug12 to nug30 as its yardstick for exact methods. nug30 resisted solution for three decades until Anstreicher, Brixius, Goux and Linderoth [2] solved it in 2000 by branch and bound on the quadratic programming bound QPB of Anstreicher and Brixius [1], on a computational grid of several hundred workstations over about a week (see [2] for the exact node count and CPU time).

QPB is a bound of moderate strength that is cheap to compute approximately by Frank–Wolfe iterations, each requiring one linear assignment problem (LAP). That balance suited a search of billions of nodes. Semidefinite relaxations are far stronger: the DNN relaxation [11, 9] is within

*Draft technical report, October 2026. Source and data: the `cognac` branch of `https://github.com/natebrix/qap`. See the acknowledgments for how this report and its software were produced.

about 3% of the optimum at the root of nug30, against about 13% for QPB (Table 2). But they have been considered too expensive to apply at more than a few nodes. Fujii et al. [4] showed that a Lagrangian-DNN branch and bound can solve nug30-sized instances with tens of thousands of nodes, on large parallel computers.

This report describes the opposite division of labor: QPB does the bulk of the search, and a certified DNN bound is applied only where QPB fails at the top of the tree, at depths 2–6. Nodes there are few, but each carries a subtree of 10^3–10^9 QPB nodes. On nug30 the DNN bound costs seconds per node and fathoms almost all of them. QPB's value at a node predicts well whether the DNN bound will fathom it.

The contributions are:

- a certificate (Section 3) that turns any ADMM multiplier into a valid lower bound with explicit floating-point safety, cheap enough to evaluate every few iterations, so that the bound can stop as soon as it fathoms;
- an implementation (Section 4) of the facially reduced ADMM with $O(m^4)$ products by the face basis and a single-precision tracked eigensolver, 13–36× faster per node than a numpy prototype;
- its integration into a QPB branch and bound (Section 5), and computational results on nug20–nug30 (Section 6), including nug30 solved in about three hours on four cores.

2 The DNN relaxation

Consider a node of the branch-and-bound tree with m facilities left to assign. Its subproblem is again a QAP (1) of size m, with reduced matrices $A, B \in \mathbb{R}^{m\times m}$, a linear term C that collects the interactions with the fixed assignments, and a constant s (the solver's `reduceQAP`). For $X \in \Pi_m$ let $x = \mathrm{vec}(X)$, ordered so that x_{ij} (facility i at location j) has lifted index $1 + im + j$, and let $y = (1, x)$ and $N = m^2 + 1$. Define the symmetric lifted cost $L \in \mathcal{S}^N$ by

$$L_{(ij),(kl)} = \tfrac{1}{2}(a_{ik}b_{jl} + a_{ki}b_{lj}), \qquad L_{0,(ij)} = L_{(ij),0} = \tfrac{1}{2}c_{ij}, \qquad L_{00} = 0. \tag{2}$$

Then for every permutation, $\langle L, yy^T\rangle = \sum a_{ik}b_{jl}x_{ij}x_{kl} + \sum c_{ij}x_{ij} = \mathrm{tr}(AXBX^T) + \mathrm{tr}(CX^T)$, the subproblem objective without s.

The face. The constraints $Xe = X^Te = e$ say that y lies in the null space of a linear map, so every lifted permutation $Y = yy^T$ has its range in a fixed subspace of dimension $(m-1)^2 + 1$. Let $V \in \mathbb{R}^{N\times K}$, $K = (m-1)^2 + 1$, be an orthonormal basis of it. Zhao et al. [11] take $V = \begin{pmatrix} 1 & 0 \\ e/m & V_m \otimes V_m \end{pmatrix}$, suitably normalized, where the columns of $V_m \in \mathbb{R}^{m\times(m-1)}$ span $e^\perp$. Every lifted permutation then satisfies $Y = VRV^T$ with $R \succeq 0$. The DNN relaxation of the subproblem is

$$\mathrm{DNN} = s + \min\{\langle L, Y\rangle : Y = VRV^T,\ R \succeq 0,\ Y_{00} = 1,\ \mathcal{G}(Y) = 0,\ 0 \le Y \le 1\}, \tag{3}$$

where $\mathcal{G}$ selects the "gangster" entries $Y_{(ij),(il)}$, $l \neq j$, and $Y_{(ij),(kj)}$, $k \neq i$, which vanish for permutations since a facility has one location and a location one facility. Oliveira, Wolkowicz

and Xu [8] solve (3) by ADMM on the splitting $Y = VRV^T$, alternating

$$R \leftarrow \mathcal{P}_+\big(V^T(Y + Z/\beta)V\big), \tag{4}$$

$$Y \leftarrow \Pi_{\mathcal{Y}}\big(VRV^T - (L + Z)/\beta\big), \tag{5}$$

$$Z \leftarrow Z + \gamma\beta\,(Y - VRV^T), \tag{6}$$

where $\mathcal{P}_+$ projects onto the positive semidefinite cone, $\mathcal{Y} = \{Y : Y_{00} = 1, \mathcal{G}(Y) = 0, 0 \le Y \le 1\}$ is a box with fixed entries (so $\Pi_{\mathcal{Y}}$ is entrywise clipping), $\beta > 0$ is the penalty and $\gamma \in (0, \frac{1+\sqrt{5}}{2})$ the dual step.

3 A certified bound from any multiplier

ADMM iterates are neither primal feasible nor dual optimal, and the primal value $\langle L, Y\rangle$ is not a bound. Branch and bound needs a number that is provably below the subproblem optimum after any number of iterations. We obtain one from the multiplier.

Lemma 1. *Every lifted permutation $Y = yy^T$ satisfies $Y = VRV^T$ with $R = V^TYV \succeq 0$ and* $\operatorname{tr}(R) = m + 1$.

Proof. y lies in the range of V, so $Y = VV^TYVV^T = VRV^T$ with $R = V^TYV = (V^Ty)(V^Ty)^T \succeq 0$, and $\operatorname{tr} R = \|V^Ty\|^2 = \|y\|^2 = 1 + \sum x_{ij}^2 = m + 1$. □

Proposition 2. *Let $Z \in \mathcal{S}^N$ be any symmetric matrix, $\lambda \ge \lambda_{\max}(V^TZV)$, and let $\mathcal{P} \supseteq \{yy^T : X \in \Pi_m\}$. Then the subproblem optimum is at least*

$$s + \min_{Y \in \mathcal{P}} \langle L + Z, Y\rangle - (m + 1)\lambda. \tag{7}$$

Proof. For a lifted permutation $Y = VRV^T$ (Lemma 1), $\langle Z, Y\rangle = \langle V^TZV, R\rangle \le \lambda_{\max}(V^TZV) \operatorname{tr} R \le (m + 1)\lambda$, since $R \succeq 0$. Hence $\langle L, Y\rangle = \langle L + Z, Y\rangle - \langle Z, Y\rangle \ge \langle L + Z, Y\rangle - (m + 1)\lambda \ge \min_{\mathcal{P}}\langle L + Z, \cdot\rangle - (m + 1)\lambda$. □

Proposition 2 holds for every Z; the quality of the bound depends on Z being near a dual optimum of (3), and on choosing Z so that λ can be small. Two choices of $\mathcal{P}$ and one of Z make it both cheap and tight.

The multiplier. The prototype of [8] projects V^TZV onto the negative semidefinite cone, which costs an extra eigendecomposition per check. Instead we use the multiplier implied by the R-step. Let $W = V^T(Y + Z/\beta)V$ be the matrix projected in (4), and

$$Z' = Z + \beta(Y - VRV^T), \qquad \text{so that} \qquad V^TZ'V = \beta(W - \mathcal{P}_+(W)) \preceq 0.$$

The last matrix is β times the negative part of W, negative semidefinite by construction. This holds even when the eigenvectors used for $\mathcal{P}_+$ are inaccurate: if $\mathcal{P}_+(W)$ is formed as W minus the computed negative part $\sum_{w_k<0} w_k v_k v_k^T$, the difference is exactly that sum. In floating point, $\lambda_{\max}(V^TZ'V)$ is a tiny positive number, which we bound rigorously below.

The polyhedral part. With $\mathcal{P}$ the box $\mathcal{Y}$ itself, $\min_{\mathcal{P}}\langle M, Y\rangle$ for $M = L + Z'$ is M_{00} plus the sum of the negative entries of M outside the gangster set: the prototype's choice. A much tighter, still cheap choice uses that every permutation is an assignment in two levels. Write $M_{(ij),\cdot}$ for row (ij) of M. For a permutation σ,

$$\langle M, yy^T\rangle = M_{00} + \sum_i \Big(2M_{0,(i\sigma_i)} + M_{(i\sigma_i),(i\sigma_i)} + \sum_{k\neq i} M_{(i\sigma_i),(k\sigma_k)}\Big). \tag{8}$$

The inner sum is a perfect matching between facilities $k \neq i$ and locations $l \neq \sigma_i$, so it is at least the value LAP_{ij} of the $(m-1)\times(m-1)$ assignment problem on the block $M_{(ij),(kl)}$, $k \neq i$, $l \neq j$. Hence

$$\min_\sigma \langle M, yy^T\rangle \;\geq\; \mathrm{GL}(M) := M_{00} + \mathrm{LAP}\big(c\big), \qquad c_{ij} = 2M_{0,(ij)} + M_{(ij),(ij)} + \mathrm{LAP}_{ij}, \tag{9}$$

a Gilmore–Lawler computation [5, 6] on the lifted matrix. It is never below the box value and reaches a given value 1.5–6× sooner in the nug20 tests. Any lower bound on each LAP_{ij} may replace it, for example the row-then-column reduction, which costs $O(m^2)$. The implementation uses the reduction, plus exact Hungarian solves when the result is within a small gap of the cutoff.

Floating point. The certified value is $s + \mathrm{GL}(L + Z') - \mu$, where the margin μ covers:

(i) $(m+1)\lambda$ with $\lambda \geq \lambda_{\max}(V^T Z' V)$, obtained by Rump's criterion [10]: if the floating-point Cholesky factorization of $\delta I - S$ succeeds for the computed $S \approx V^T Z' V$, then $\lambda_{\max}(S) \leq \delta + c$ with an explicit c depending on the dimension, the unit roundoff u and $\mathrm{tr}(\delta I - S)$. To this is added $64\, m\, u\, \|Z'\|_F$ for the error of computing S through Householder transforms rather than with an exact V.

(ii) the summation and rounding errors of (9), accumulated in extended precision: $8(m+2)u\,\|M\|_1$, plus $u\sum|L|$ for the objective scaling.

These are standard error bounds with generous constants, not interval arithmetic. At $m = 28$ the largest margin seen was about 10^{-7} in objective units, against integer objective values. The solver fathoms only when the certified value exceeds the incumbent minus 1.95 for instances whose objective values are provably even (Section 5).

Remark 3. Nothing in the certificate depends on the ADMM having converged, on the accuracy of its eigenvectors, or on the single-precision arithmetic used in the R-step (Section 4). Only the multiplier Z', the Cholesky test and (9) enter the certified value, all in double precision.

4 Implementation

The bound is implemented in C++ (`latest/dnn.C`, about 1100 lines) with LAPACK and BLAS (OpenBLAS). A numpy prototype (`experiments/dnn_bound`) was ported faithfully; a prototype mode reproduces its certified values to 10^{-7}.

Products with V. The prototype forms V explicitly and computes $V^T M V$ and $V R V^T$ with dense products of order $N \times N$, which costs $O(m^6)$. We take for V_m the first $m-1$ columns of the Householder reflector H with $He_{m-1} = e/\sqrt{m}$. Then $V^T M V$ and $V R V^T$ are a reflection along each of the four tensor modes plus a 2×2 rotation coupling the constant coordinate, $O(m^4)$ per product. Both the R-step and the certificate are invariant under an orthogonal change of basis of the face, so any orthonormal V gives the same iterates and bound.

Table 1: The certified DNN bound in C++ against the numpy prototype, CPU seconds per node, cutoff = optimum -1.95 (one core; prototype times from the studies in `experiments/dnn_bound`).

node set	fathomed (both)	C++	prototype	ratio
nug20, depth 2 (all 98, $m = 18$)	96	0.62	8.35	13×
nug30, depth 2 (36 random, $m = 28$)	34	9.6	139	14.5×
nug30, depth 2 (8 lowest QPB)	0	11 (200 it.)	397	36×
nug30, depth 3 (20, $m = 27$)	19	7.4	121	16×

Eigensolver. Each R-step needs the projection of a $K \times K$ symmetric matrix ($K = 730$ at $m = 28$) with hundreds of eigenvalues of each sign, so a partial spectrum does not help directly. Between iterations the matrix changes little. We track its negative eigenspace by a Chebyshev-filtered subspace iteration in single precision (BLAS level 3), with a full single-precision eigendecomposition every 20 iterations. Because R is formed as W minus the computed negative part, the certificate is unaffected (Section 3). When a check fails the Cholesky test, or the semidefinite penalty alone decides whether the node is fathomed, the multiplier is re-projected in double precision before certifying.

Parameters. The objective is scaled to $\max |L| = 1$. We use $\beta = 0.3 \cdot m/3$ (the prototype's $m/3$ times 0.3), which certifies 2–3× sooner on hard nodes, and $\gamma = 1.618$. A certificate is computed every 10 iterations, and the run stops at the first certified value above the cutoff. An optional give-up rule stops a node early when its certified value, extrapolated generously from its recent gains, stays below the cutoff. This only affects cost: the node is then branched on as usual.

Speed. Table 1 compares the C++ bound with the prototype on the same nodes. Measured back to back on a loaded 4-core Xeon, an ADMM iteration at $m = 28$ costs 43 ms against 339 ms. Per node, the C++ bound needs fewer iterations to certify, thanks to the tighter polyhedral part and the smaller β, and is 13–36× faster.

5 Use in branch and bound

The branch and bound is the QPB code of [1, 2], with improvements made since then: Frank–Wolfe warm-started from the parent's solution, keeping the best (bound, reduced cost) pair over all iterations, and cheaper, corrected lookahead branching at the top levels. For grid instances, it also uses a branching rule on the most eccentric free location down to depth 6 (`PARAM/nug_deep.par`). The DNN bound is added as follows.

1. At a node of depth $d \in [d_1, d_2]$ whose QPB value does not fathom it, the certified DNN bound is computed for its reduced subproblem (`bnbdrv -x d1:d2[:gap[:giveup]]`).

2. If it exceeds the incumbent minus the fathoming tolerance, the node is fathomed. Otherwise it only raises the node's recorded bound.

3. Branching, and the elimination of children by bound $+ U_{ij} \geq$ incumbent, use the QPB value and its reduced costs U, never the DNN value. The elimination argument is a property of the Frank–Wolfe linearization of QPB.

Table 2: Root bounds and DNN fathoming at the top of the tree (prototype studies; QPB from the solver; incumbent = optimum).

	nug12	nug20	nug30
optimum	578	2570	6124
root QPB	478.5	2243.9	5332.3
root DNN (certified)	568.0	2506.3	5949.3
depth 1: fathomed / QPB-open	–	1 / 6	0 / 1 sampled
depth 2: fathomed / QPB-open	–	96 / 98	34 / 36 sampled
depth 3: fathomed / QPB-open	–	200 / 200 sampled	19 / 20 sampled

The fathoming tolerance is 1.95 when all objective values are provably even, as for the nug instances, 0.95 when they are integers, and 0 otherwise. Search statistics, including DNN counts, are kept in the checkpoint files, so a run can be stopped, resumed and split across processes (`bench/parallel.py`) with identical totals. One caveat: the DNN bound's floating-point results depend on the BLAS kernels selected for the CPU, so node counts with the DNN bound can differ between machines if a certified value lands within about 10^{-6} of a cutoff. Validity is not affected.

Where to apply it. Two studies with the prototype measured the bound on QPB-open nodes (Table 2). At depth 1 it rarely fathoms. At depth 2 it fathoms nearly all nodes of nug20 and 94% of a random sample of nug30's. On nug30 the outcome is almost a function of the node's QPB value: every sampled node with QPB below about 5548 was missed, every node above about 5554 fathomed. At depth 3, under the depth-2 nodes it misses, it fathoms almost everything. The misses are the most expensive subtrees, so the bound has to be applied at several consecutive depths. In the full solves, depths 2 to 5 or 6 worked best.

6 Computational results

All runs use the known optimum as the incumbent, as in [2], so they measure the work of the proof. They ran on a cloud virtual machine with 4 cores (Intel Xeon, 2.1–2.8 GHz), with `bench/parallel.py` distributing subtrees to 4 solver processes. CPU is the sum over processes.

Table 3 summarizes the solves. At nug20 the DNN bound at depths 2–3 removes all but 141 of 526,205 nodes and halves the time. At nug24 the tree collapses to 299 nodes. From nug25 on, the DNN bound does most of the work: on nug28, 12,926 of 12,976 CPU seconds are spent in it, on 1,727 nodes of which 1,664 are fathomed.

nug30. The first solve applied the DNN bound at depths 2–5. QPB search below the depth-5 nodes the bound missed grew to about 12,000 CPU seconds, so for the remaining work (506 open nodes, including a whole depth-1 subtree) the bound was extended to depth 6, which finished in 21 minutes. In total, 45,486,332 nodes and 40,555 CPU seconds. The DNN bound ran on 5,615 nodes, fathomed 5,403, and accounted for 26,770 of those seconds. A clean run with the DNN bound at depths 2–6 throughout took **[pending]** nodes, **[pending]** CPU seconds and **[pending]** of wall time (**[pending]**).

Table 3: Branch and bound with and without the DNN bound (4 workers unless noted). QPB only: default parameters. DNN: `-x d1:d2`, giving up on hopeless nodes after 300 iterations where shown (:300).

instance	bounds	nodes	CPU s	wall
nug20	QPB only (1 core)	526,205	197	–
nug20	QPB + DNN 2–3 (1 core)	141	105	–
nug24	QPB only	≈25 M	≈5,000	≈25 min
nug24	QPB + DNN 2–3	299	796	4.4 min
nug25	QPB only	52,989,998	18,353	78 min
nug25	QPB (`nug_deep`) + DNN 2–5	717	3,279	15 min
nug25	QPB (`nug_deep`) + DNN 2–5:300	761	1,447	6.2 min
nug27	QPB (`nug_deep`) + DNN 2–5	675	4,428	20 min
nug28	QPB (`nug_deep`) + DNN 2–5	1,736	12,976	62 min
nug30	QPB (`nug_deep`) + DNN 2–5:300, then 2–6:300	45,486,332	40,555	≈3.0 h
nug30	QPB (`nug_deep`) + DNN 2–6:300	**[pending]**	**[pending]**	**[pending]**

7 Validation

A branch-and-bound proof is only as good as its bounds, so the DNN bound was checked in several independent ways.

- **Exhaustive checks.** 1,512 certified bounds on random instances and subproblems with $m \leq 8$ (several families, including linear terms and ties), each compared with the optimum found by enumeration. No bound exceeds the optimum, and no fathom is ever claimed with the cutoff at the optimum.
- **Inside the search.** The solver's brute-force test (4,536 runs on random instances with $n \leq 8$, eight parameter sets, three upper-bound modes) passes with the DNN bound applied at every depth, with and without the give-up rule.
- **Against the prototype.** In prototype mode the C++ bound reproduces the numpy prototype's certified values at 44 checkpoints to a relative $8.6 \cdot 10^{-8}$. On nug20 nodes whose subproblems were solved exactly, the bounds are below the optima and fathom the same nodes.
- **Reproducibility.** Results are bitwise identical across runs on one machine. Valgrind reports no memory errors on a debug build. Serial, resumed and parallel runs give identical node counts.
- **Independent review. [pending: results of the mathematical and code reviews, and of the adversarial tests.]**

8 Discussion

Twenty-six years ago nug30 needed QPB's cheapness: billions of nodes, each bounded in milliseconds, on hundreds of machines. Today the same search needs only a few thousand much stronger bounds at the top, and QPB below them, to finish on one small machine in an afternoon. Hardware

accounts for part of the gain, but most of it comes from the division of labor. The stronger bound, certified at every step, is spent only where a single fathom removes a large subtree.

Several directions remain. The give-up rule and the depths of application were tuned by hand on nug25. A QPB threshold predicts the DNN outcome almost perfectly, which suggests skipping nodes the bound will miss. Warm-starting a child's ADMM from its parent's multiplier cut iterations 2–2.5× in early tests. And instances beyond nug30 (tai35, tho40, and the Eschermann and Taillard families) are natural next targets.

Acknowledgments

The software described here, the experiments and a first draft of this report were produced with Claude, an AI system by Anthropic, working in the author's repository under the author's direction, with independent AI agents used for the prototype studies, the C++ port and the reviews. Jean-Pierre Goux contributed the "nug30 Laboratory" study whose ideas started this work.

References

[1] K.M. Anstreicher and N.W. Brixius, A new bound for the quadratic assignment problem based on convex quadratic programming, *Mathematical Programming* 89 (2001) 341–357.

[2] K. Anstreicher, N. Brixius, J.-P. Goux and J. Linderoth, Solving large quadratic assignment problems on computational grids, *Mathematical Programming* 91 (2002) 563–588.

[3] R.E. Burkard, S.E. Karisch and F. Rendl, QAPLIB – a quadratic assignment problem library, *Journal of Global Optimization* 10 (1997) 391–403.

[4] K. Fujii, N. Ito, S. Kim, M. Kojima, Y. Shinano and K.-C. Toh, Solving challenging large scale QAPs, arXiv:2101.09629 (2021).

[5] P.C. Gilmore, Optimal and suboptimal algorithms for the quadratic assignment problem, *SIAM Journal on Applied Mathematics* 10 (1962) 305–313.

[6] E.L. Lawler, The quadratic assignment problem, *Management Science* 9 (1963) 586–599.

[7] C.E. Nugent, T.E. Vollmann and J. Ruml, An experimental comparison of techniques for the assignment of facilities to locations, *Operations Research* 16 (1968) 150–173.

[8] D.E. Oliveira, H. Wolkowicz and Y. Xu, ADMM for the SDP relaxation of the QAP, *Mathematical Programming Computation* 10 (2018) 631–658.

[9] J. Povh and F. Rendl, Copositive and semidefinite relaxations of the quadratic assignment problem, *Discrete Optimization* 6 (2009) 231–241.

[10] S.M. Rump, Verification of positive definiteness, *BIT Numerical Mathematics* 46 (2006) 433–452.

[11] Q. Zhao, S.E. Karisch, F. Rendl and H. Wolkowicz, Semidefinite programming relaxations for the quadratic assignment problem, *Journal of Combinatorial Optimization* 2 (1998) 71–109.